

Badania Operacyjne

Informatyka 2025/2026

Planer tras turystycznych — optymalizacja trasy

z wykorzystaniem algorytmu *Artificial Bee Colony*

Skład zespołu:

Bartosz Tokarz
Adrian Michalski
Mikołaj Kaleta

18 maja 2026

Streszczenie

Projekt dotyczy zadania planowania trasy turystycznej na siatce komórek z ważonymi przejściami, ograniczeniami budżetowymi, czasowymi oraz minimalną i maksymalną liczbą atrakcji każdego typu. Dla wygenerowania populacji rozwiązań dopuszczalnych zaproponowano i zaimplementowano algorytm *Artificial Bee Colony* (ABC), w którym rozwiązanie reprezentowane jest jako lista *waypointów* (start, atrakcje w kolejności, koniec) łączonych krokami A^* na siatce 8-kierunkowej z twardym zakazem powtarzania krawędzi.

Spis treści

1	Wstęp	2
2	Opis zagadnienia	3
2.1	Sformułowanie problemu	3
2.2	Model matematyczny	4
2.2.1	Definicja grafu	4
2.2.2	Koszt przejścia	4
2.2.3	Atrakcje	4
2.2.4	Typy atrakcji	4
2.2.5	Zmienne decyzyjne	4
2.2.6	Funkcja celu	5
2.2.7	Ograniczenia	5
2.3	Uwagi modelowe i decyzje projektowe	6
3	Opis algorytmów	6
3.1	Artificial Bee Colony (ABC)	6
3.1.1	Reprezentacja rozwiązania	7
3.1.2	Wygenerowanie populacji początkowej	8
3.1.3	Operatory modyfikacji rozwiązania (sąsiedztwo)	9
3.1.4	Selekcja typu onlooker (wagi rankingowe)	9
3.1.5	Faza zwiadowcza (scout)	10
3.1.6	Pseudokod operatora mutacji	11
3.1.7	Funkcja oceny ścieżki	11
3.2	Podejścia pomocnicze	12
3.2.1	Random Walk	12
3.2.2	Greedy attractions	12
3.3	Inne pomysły rozważane w fazie projektowej	12

4	Aplikacja — dokumentacja użytkownika	12
4.1	Wymagania i instalacja	13
4.2	Charakterystyka danych	13
4.2.1	Format mapy — CLI (<code>map.json</code>)	13
4.2.2	Format mapy — aplikacja web (<code>scenarios/*.json</code>)	13
4.2.3	Format populacji (<code>initial_population.json</code>)	14
4.3	Uruchamianie — tryb CLI	14
4.3.1	Krok 1. Wygenerowanie mapy	14
4.3.2	Krok 2. Generowanie populacji metodą ABC	14
4.3.3	Krok 3. Walidacja wygenerowanej populacji	15
4.3.4	Krok 4. Wizualizacja tras	15
4.4	Uruchamianie — aplikacja web (Streamlit)	15
4.4.1	Tryb „Edytor / Generator”	16
4.4.2	Tryb „Wyniki”	16
4.5	Ustawianie parametrów instancji	16
4.6	Ustawianie parametrów algorytmu (ABC)	16
4.7	Wyniki i sposób ich interpretacji	17
5	Eksperymenty	17
5.1	Plan eksperymentalny	17
5.2	Porównanie z baselineami	18
5.3	Zbieranie wyników	18
5.4	Wyniki wstępne	18
6	Podsumowanie	19
6.1	Wnioski	19
6.2	Napotkane problemy	19
6.3	Kierunki rozwoju	19
7	Literatura	20
8	Dodatek — podział pracy	20

1 Wstęp

Cel projektu

Celem projektu jest opracowanie narzędzia wspomagającego planowanie trasy turystycznej na zadanej mapie. Turysta wyrusza z ustalonego punktu *start*, porusza się po siatce komórek o zróżnicowanych kosztach przejścia i ma za zadanie zakończyć podróż w punkcie *end*, odwiedzając po drodze atrakcje w taki sposób, aby zmaksymalizować łączną wartość zdobytych atrakcji przy jednoczesnym spełnieniu ograniczeń:

- budżetowego (łączny koszt biletów wstępu do odwiedzonych atrakcji nie może przekroczyć założonego budżetu),
- czasowego (łączny czas: ruch po siatce plus czas zwiedzania nie może przekroczyć dostępnego budżetu czasu),
- dotyczącego liczności typów atrakcji (dla każdego typu zdefiniowane są minimalna i maksymalna liczba atrakcji, które trzeba lub można odwiedzić).

Dodatkowo, ze względów modelowych, przyjęto twarde wymaganie, aby trasa *nie używała dwukrotnie tej samej krawędzi* (przejścia między dwoma sąsiednimi polami) — co odpowiada intuicji, że turysta nie chce przechodzić tą samą wąską ścieżką w obie strony.

Założenia projektu

1. Mapa jest dyskretną siatką $H \times W$ komórek. W każdej komórce (r, c) zdefiniowana jest waga $w_{r,c} \in \mathbb{N}_+$ reprezentująca koszt „bycia” w tej komórce (im wyższa waga, tym trudniejszy/dłuższy teren).
2. Ruch w siatce odbywa się w jednym z ośmiu kierunków (4 osiowe + 4 diagonalne). Koszt przejścia z komórki (r, c) do sąsiada wynosi $w_{r,c}$ pomnożone przez 1,0 dla ruchu osiowego oraz 1,4 dla ruchu diagonalnego (przybliżenie $\sqrt{2}$).
3. Atrakcje to wyróżnione komórki o zadanej wartości, koszcie biletu, czasie zwiedzania i typie. Wizyta polega na wejściu na komórkę atrakcji; ponowne odwiedzenie tej samej atrakcji nie daje żadnych dodatkowych punktów.
4. Algorytm operuje w paradygmacie *generowania populacji rozwiązań dopuszczalnych* — każde rozwiązanie w populacji końcowej musi spełniać wszystkie ograniczenia (budżet, czas, liczności typów, brak powtórzonych krawędzi).
5. Komponentem dostarczającym rozwiązanie początkowe i jednocześnie mechanizmem poszukiwania jest meta-heurystyka *Artificial Bee Colony* (ABC).
6. Aplikacja składa się z modułów: `map_generator` (generator instancji), `parser/load_json` (obsługa formatu danych), `abc_algorithm` (rdzeń algorytmu), `initial_population` (skrypt uruchomieniowy generujący populację) oraz `visualize_paths` (wizualizacja). Dodatkowo dostępna jest aplikacja webowa (`app.py`, Streamlit) służąca jako interaktywny edytor mapy.

Zakres dokumentu

Dokument prezentuje formalny opis problemu i jego model matematyczny (rozdział 2), szczegółowo omawia zaprojektowaną metodę rozwiązania (rozdział 3), opis aplikacji wraz z instrukcją uruchomienia (rozdział 4), zarys planowanych eksperymentów (rozdział 5), wnioski i kierunki rozwoju (rozdział 6) oraz literaturę i dodatek z podziałem prac.

2 Opis zagadnienia

2.1 Sformułowanie problemu

Środowisko modelowane jest jako siatka, gdzie każde pole reprezentuje fragment terenu o określonym koszcie przejścia.

- Każde pole posiada wagę określającą trudność terenu.
- Koszt przejścia zależy od kierunku ruchu:
 - ruch prosty (poziomy/pionowy): mnożnik 1,
 - ruch po skosie (diagonalny): mnożnik 1,4.

Na planszy rozmieszczone są **atrakcje turystyczne**:

- każda atrakcja ma wartość turystyczną,
- każda atrakcja ma koszt odwiedzenia (np. cena biletu),
- każda atrakcja należy do określonego *typu* (zabytek, przyroda, restauracja itp.).

Podróżnik:

- startuje w punkcie s ,
- kończy w punkcie t ,
- posiada ograniczony czas T oraz budżet B .

Dodatkowe wymagania konstrukcyjne:

- każdego przejścia (krawędzi w siatce) można użyć tylko raz,
- każda atrakcja może być odwiedzona maksymalnie raz,
- dla każdego typu atrakcji zdefiniowane są limity dolny l_τ i górny u_τ na liczbę atrakcji tego typu w trasie.

Problem stanowi rozszerzenie klasycznego zagadnienia znajdowania ścieżki o elementy decyzyjne związane z wyborem punktów pośrednich — bliski *Orienteering Problem* z ograniczeniami typowymi.

2.2 Model matematyczny**2.2.1 Definicja grafu**

Problem reprezentowany jest jako graf:

$$G = (V, E)$$

gdzie:

- V — zbiór pól siatki,
- E — możliwe przejścia (krawędzie) między polami sąsiadującymi w siatce 8-kierunkowej (sąsiedztwo Moore'a).

2.2.2 Koszt przejścia

Koszt przejścia pomiędzy polami:

$$c_{ij} = w_i \cdot d_{ij}$$

gdzie:

- w_i — waga pola i ,
- $d_{ij} \in \{1, 1,4\}$ — współczynnik kierunku ruchu (1 dla ruchu osiowego, 1,4 dla diagonalnego).

2.2.3 Atrakcje

Niech:

- $A \subseteq V$ — zbiór atrakcji,
- v_i — wartość atrakcji i ,
- p_i — koszt (cena) atrakcji i ,
- $\tau_i \in \mathcal{T}$ — typ atrakcji i .

2.2.4 Typy atrakcji

Niech $\mathcal{T}$ oznacza zbiór wszystkich typów atrakcji. Każdemu typowi $\tau \in \mathcal{T}$ przypisane są następujące parametry:

- t_τ — czas potrzebny na zwiedzenie atrakcji typu τ ,
- l_τ — minimalna liczba atrakcji typu τ , które należy odwiedzić,
- u_τ — maksymalna liczba atrakcji typu τ , które można odwiedzić.

2.2.5 Zmienne decyzyjne

$$x_{ij} = \begin{cases} 1 & \text{jeśli przechodzimy z } i \text{ do } j, \\ 0 & \text{w przeciwnym razie,} \end{cases} \quad y_i = \begin{cases} 1 & \text{jeśli odwiedzamy atrakcję } i, \\ 0 & \text{w przeciwnym razie.} \end{cases}$$

2.2.6 Funkcja celu

Celem jest maksymalizacja wartości odwiedzonych atrakcji:

$$\max \sum_{i \in A} v_i \cdot y_i. \quad (1)$$

2.2.7 Ograniczenia

1. Start:

$$\sum_j x_{sj} = 1. \quad (2)$$

2. Koniec:

$$\sum_i x_{it} = 1. \quad (3)$$

3. Zachowanie przepływu:

$$\sum_i x_{ik} = \sum_j x_{kj} \quad \forall k \in V \setminus \{s, t\}. \quad (4)$$

4. Ograniczenie czasu:

$$\sum_{(i,j) \in E} c_{ij} \cdot x_{ij} + \sum_{i \in A} y_i \cdot t_{\tau_i} \leq T. \quad (5)$$

5. Ograniczenie budżetu:

$$\sum_{i \in A} p_i \cdot y_i \leq B. \quad (6)$$

6. Ograniczenia ilościowe na typy atrakcji:

$$l_\tau \leq \sum_{\substack{i \in A \\ \tau_i = \tau}} y_i \leq u_\tau \quad \forall \tau \in \mathcal{T}. \quad (7)$$

7. Powiązanie atrakcji ze ścieżką. Atrakcja może być odwiedzona tylko wtedy, gdy ścieżka przez nią przechodzi:

$$y_i \leq \sum_j x_{ji} \quad \forall i \in A. \quad (8)$$

8. Jednokrotność użycia krawędzi i odwiedzin atrakcji:

$$x_{ij} + x_{ji} \leq 1 \quad \forall (i, j) \in E \quad (\text{krawędź nieskierowana użyta co najwyżej raz}) \quad (9)$$

$$y_i \leq 1 \quad \forall i \in A \quad (\text{atrakcja odwiedzona co najwyżej raz}) \quad (10)$$

9. Binarność:

$$x_{ij} \in \{0, 1\}, \quad y_i \in \{0, 1\}. \quad (11)$$

Równania (1)–(11) formalnie definiują problem optymalizacji jako mieszany model przepływu w grafie z elementami doboru podzbioru (klasy *Orienteering Problem* z ograniczeniami typowymi i zakazem powtórzeń krawędzi).

2.3 Uwagi modelowe i decyzje projektowe

- **Złożoność.** Problem jest NP-trudny (zawiera w sobie uogólnione *Orienteering Problem*), co uzasadnia zastosowanie metod heurystycznych i meta-heurystycznych — w tym projekcie algorytmu *Artificial Bee Colony* (patrz rozdział 3).
- **Ścieżka jako konstrukcja pomocnicza.** W implementacji fragmenty trasy pomiędzy waypointami (start, kolejne atrakcje, koniec) generowane są algorytmem A^* (najtańsza ścieżka, koszt zgodny z c_{ij} , sąsiedztwo 8-kierunkowe). Algorytm uwzględnia dynamiczne zakazy: blokowane są pola atrakcji niezaplanowanych w bieżącej liście waypointów, a także krawędzie wcześniej zużyte (twardy zakaz powtórzeń krawędzi, (9)).
- **Atrakcje są blokujące.** W implementacji pole atrakcji, której nie planujemy odwiedzić, jest blokowane dla ścieżki — jeśli komórka atrakcji znajdzie się na trasie, atrakcja zostanie „zaliczona”. Wymusza to spójność: trasa nie biegnie tranzytem przez atrakcje, których nie zamierzaliśmy odwiedzić.
- **Kara vs. dopuszczalność.** W zaproponowanym podejściu nie stosujemy funkcji kary za naruszenie ograniczeń: algorytm utrzymuje jedynie rozwiązania w pełni dopuszczalne (spełniające (2)–(11)), a rozwiązania niedopuszczalne są odrzucane już na etapie konstrukcji.
- **Kryterium porównawcze.** Do porównywania rozwiązań w populacji ABC stosowana jest leksykograficzna krotka

$$(\sum v_i y_i, -(\text{czas trasy}), -(\text{koszt ruchu}), -(\text{koszt atrakcji})),$$

gdzie minus oznacza preferencję niższej wartości. Pozwala to przy jednakowej wartości celu różnicować rozwiązania krótsze czasowo i tańsze, co ułatwia generowanie zróżnicowanej populacji.

3 Opis algorytmów

W ramach projektu rozważano i częściowo zaimplementowano kilka podejść do generowania trasy. Główną meta-heurystyką, która została w pełni zaimplementowana i służy do generowania populacji rozwiązań dopuszczalnych, jest *Artificial Bee Colony* (ABC). Dodatkowo, jako prosty baseline porównawczy i mechanizm testowania interfejsu, zaimplementowano dwie pomocnicze procedury: *Random Walk* oraz *Greedy attractions* (zachłanne dążenie do najbliższej atrakcji).

3.1 Artificial Bee Colony (ABC)

ABC to populacyjna meta-heurystyka inspirowana zachowaniem pszczół. W klasycznej formie wyróżnia trzy typy „pszczół”:

- **Pszczoły zatrudnione** (*employed bees*) — przypisane do konkretnych rozwiązań (źródeł pokarmu) i modyfikujące je lokalnie.
- **Obserwatorzy** (*onlookers*) — wybierają źródło z prawdopodobieństwem rosnącym z jakością źródła i również je modyfikują.
- **Zwiadowcy** (*scouts*) — zastępują źródła, które od zbyt wielu prób (*trial*) nie poprawiły się, nowymi losowymi rozwiązaniami.

Schemat iteracji ABC dla naszego problemu przedstawia Algorithm 1.

Algorytm 1 Generowanie populacji metodą ABC (ogólny zarys)

Wejście: dane wejściowe `parsed_data`, rozmiar populacji N , liczba iteracji I , limit stagnacji L , liczba onlookerów O

Wyjście: populacja $\mathcal{P} = \{s_1, \dots, s_N\}$ rozwiązań dopuszczalnych

```

1:  $\mathcal{P} \leftarrow \emptyset$ 
2: dopóki  $|\mathcal{P}| < N$  wykonuj
3:    $s \leftarrow \text{LOSOWEROZWIAZANIEDOPUSZCZALNE}$ 
4:   jeżeli  $s \neq \perp$  i  $s$  nie jest duplikatem w  $\mathcal{P}$  wtedy
5:      $\mathcal{P} \leftarrow \mathcal{P} \cup \{s\}$ 
6:   koniec jeżeli
7: koniec dopóki
8: dla  $it = 1, \dots, I$  wykonuj
9:   dla  $i = 1, \dots, N$  wykonuj ▷ Faza employed
10:     $s' \leftarrow \text{MUTACJAWAYPOINTOW}(\mathcal{P}_i)$ 
11:    jeżeli  $s' \neq \perp$  i  $s' \succ \mathcal{P}_i$  wtedy
12:       $\mathcal{P}_i \leftarrow s'$ ;  $\mathcal{P}_i.\text{trial} \leftarrow 0$ 
13:    w przeciwnym razie
14:       $\mathcal{P}_i.\text{trial} \leftarrow \mathcal{P}_i.\text{trial} + 1$ 
15:    koniec jeżeli
16:  koniec dla
17:   $w \leftarrow \text{WAGIRANKINGOWE}(\mathcal{P})$ 
18:  dla  $k = 1, \dots, O$  wykonuj ▷ Faza onlooker
19:     $i \leftarrow \text{WYLOSUJZWAGAMI}(w)$ 
20:     $s' \leftarrow \text{MUTACJAWAYPOINTOW}(\mathcal{P}_i)$ 
21:    jeżeli  $s' \neq \perp$  i  $s' \succ \mathcal{P}_i$  wtedy
22:       $\mathcal{P}_i \leftarrow s'$ ;  $\mathcal{P}_i.\text{trial} \leftarrow 0$ 
23:    w przeciwnym razie
24:       $\mathcal{P}_i.\text{trial} \leftarrow \mathcal{P}_i.\text{trial} + 1$ 
25:    koniec jeżeli
26:  koniec dla
27:  dla  $i = 1, \dots, N$  wykonuj ▷ Faza scout
28:    jeżeli  $\mathcal{P}_i.\text{trial} \geq L$  wtedy
29:       $\mathcal{P}_i \leftarrow \text{LOSOWEROZWIAZANIEDOPUSZCZALNE}$ 
30:       $\mathcal{P}_i.\text{trial} \leftarrow 0$ 
31:    koniec jeżeli
32:  koniec dla
33: koniec dla
34: zwróć  $\mathcal{P}$ 

```

Relacja porządku $s' \succ s$ to leksykograficzne porównanie krotek:

$$(\text{Value}(s'), -\text{Time}(s'), -\text{Cost}_{\text{ruch}}(s'), -\text{Cost}_{\text{atrakcji}}(s')) > (\text{Value}(s), -\text{Time}(s), -\text{Cost}_{\text{ruch}}(s), -\text{Cost}_{\text{atrakcji}}(s))$$

3.1.1 Reprezentacja rozwiązania

Rozwiązanie w populacji jest dwupoziomowe:

1. **Plan logiczny** — lista *waypointów* $\mathbf{W} = \langle s, a_{i_1}, a_{i_2}, \dots, a_{i_k}, e \rangle$, gdzie s, e to punkty startu i końca, a a_{i_j} to wybrane atrakcje w zaplanowanej kolejności odwiedzenia.
2. **Realizacja fizyczna** — konkretna ścieżka komórek $P = \langle p_0, p_1, \dots, p_L \rangle$, zbudowana z planu logicznego przez łączenie kolejnych waypointów najtańszą ścieżką A^* z uwzględnieniem zakazu używania już wykorzystanych krawędzi i niezaplanowanych komórek atrakcji.

Każde rozwiązanie w populacji przechowuje również:

- **path** — pełną listę komórek (lista par (r, c)),
- **visited_attractions** — listę odwiedzonych atrakcji,
- **movement_cost**, **attraction_cost**, **total_time**, **total_value**,
- **type_counts** — wektor liczności atrakcji każdego typu,
- flagi dopuszczalności: **budget_ok**, **time_ok**, **type_ok**, **feasible**,
- **trial** — licznik nieudanych modyfikacji (na potrzeby zwiadowców),
- **id**, **waypoints** — identyfikator i plan logiczny.

3.1.2 Wygenerowanie populacji początkowej

Populacja początkowa składa się z N losowych rozwiązań **dopuszczalnych**. Procedura konstrukcji pojedynczego rozwiązania składa się z dwóch kroków:

1. Losowy wybór listy waypointów spełniający ograniczenia liczności typów (Algorithm 2).
2. Materializacja ścieżki przez łączenie kolejnych waypointów algorytmem A^* z zakazem powtarzania krawędzi i komórek (Algorithm 3). Jeżeli ścieżka się nie udaje (np. nie ma fizycznej drogi spełniającej ograniczenia), rozwiązanie jest odrzucane i próbujemy następne.

Algorytm 2 LOSOWEWAYPOINTY — losowanie planu atrakcji

Wejście: dane mapy, punkty s, e , parametr $E_{\max}$ (maksymalna liczba dodatkowych atrakcji)

Wyjście: lista waypointów **W** lub $\perp$ przy braku spełnienia minimów

```

1: chosen  $\leftarrow \emptyset$ 
2: dla każdego typ  $t$  wykonuj
3:    $n_t \leftarrow m_t$  — liczba atrakcji typu  $t$  obecnych w  $\{s, e\}$ 
4:   jeżeli  $n_t > 0$  wtedy
5:     wylosuj  $n_t$  atrakcji typu  $t$  spośród dostępnych
6:     jeżeli kandydatów jest za mało wtedy
7:       zwróć  $\perp$ 
8:     koniec jeżeli
9:     dołącz wylosowane do chosen
10:  koniec jeżeli
11: koniec dla
12:  $E \leftarrow \text{rand}(0, \min(E_{\max}, |\text{wolnych atrakcji}|))$ 
13: dla  $k = 1, \dots, E$  wykonuj
14:   wylosuj atrakcję  $a$  z prawdopodobieństwem ważonym  $\frac{v_a+1}{\tau_{ta}+1}$ 
15:   jeżeli  $a \notin \text{chosen}$  i dorzucenie nie łamie  $M_{ta}$  wtedy
16:     chosen  $\leftarrow \text{chosen} \cup \{a\}$ 
17:   koniec jeżeli
18: koniec dla
19: pomieszaj kolejność chosen
20: zwróć konkatencja  $\langle s \rangle, \text{chosen}, \langle e \rangle$ 
```

Algorytm 3 BUDUJSCIEZKEZWAYPOINTOW — konstrukcja ścieżki fizycznej**Wejście:** mapa, waypointy $\mathbf{W} = \langle w_0, w_1, \dots, w_K \rangle$ **Wyjście:** ścieżka P (lista komórek) lub $\perp$ jeśli nie istnieje

```

1: visited  $\leftarrow \{w_0\}$ ;   usedEdges  $\leftarrow \emptyset$ 
2:  $P \leftarrow \langle w_0 \rangle$ 
3: dla  $i = 0, \dots, K - 1$  wykonuj
4:   blokady  $\leftarrow (\mathcal{A}\text{-pozycje} \setminus \{w_0, w_K, w_i, w_{i+1}\}) \cup (\text{visited} \setminus \{w_i, w_{i+1}\})$ 
5:    $S \leftarrow \text{ASTAR}(w_i, w_{i+1}, \text{blokady}, \text{usedEdges})$ 
6:   jeżeli  $S = \perp$  wtedy zwróć  $\perp$ 
7:   koniec jeżeli
8:   dla każda kolejna krawędź  $(u, v)$  w  $S$  wykonuj
9:     usedEdges  $\leftarrow \text{usedEdges} \cup \{\{u, v\}\}$ 
10:  koniec dla
11:  dla każdy kolejny węzeł  $p$  w  $S$  poza pierwszym wykonuj
12:    jeżeli  $p \in \text{visited}$  wtedy zwróć  $\perp$ 
13:    koniec jeżeli
14:    visited  $\leftarrow \text{visited} \cup \{p\}$ ; dołącz  $p$  do  $P$ 
15:  koniec dla
16: koniec dla
17: zwróć  $P$ 

```

Procedura A^* jest standardowa — używa heurystyki optymistycznej $h(p, g) = w_{\min} \cdot (1,4 \cdot d_{\text{diag}} + d_{\text{ax}})$, gdzie $w_{\min}$ to minimalna waga na siatce, d_{diag} to liczba kroków diagonalnych, a d_{ax} to liczba kroków osiowych w ruchu Chebysheva. Zapewnia to dopuszczalność heurystyki i optymalność znalezionej trasy w sensie kosztu ruchu (przy zadanych blokadach komórek i zakazach krawędzi).

3.1.3 Operatory modyfikacji rozwiązania (sąsiedztwo)

Modyfikacja rozwiązania (MUTACJAWAYPOINTOW) polega na zmianie listy waypointów, a następnie próbie materializacji nowej ścieżki. Wybór operatora jest losowy z rozkładem przedstawionym w Table 1. W każdej próbie sprawdzane są ograniczenia liczności typów; jeżeli kandydat nie da się zmaterializować lub nie spełnia ograniczeń, próba kończy się fiaskiem.

Tabela 1: Operatory mutacji listy waypointów (sąsiedztwo).

Operator	P	Opis
Dodaj atrakcję	0,30	Wybierany kandydat $a \notin \mathbf{W}$ ze zbioru 20 najlepiej rokujących wg $(v_a + 1)/(\tau_{t_a} + 1)$; wstawiany w losowej pozycji, jeśli nie łamie M_{t_a} .
Usuń atrakcję	0,25	Losowo wybierany element $\mathbf{W}$, którego usunięcie nie złamie m_t .
Zamień sąsiadujące	0,20	Wybierane dwie pozycje w $\mathbf{W}$ i zamieniane miejscami.
2-opt (odwróć fragment)	0,20	Losowo wybierany fragment $[i, j]$ i odwracany.
Przesuń atrakcję	0,05	Wyjęcie pojedynczego elementu i wstawienie w innym miejscu (<i>shift/relocate</i>).

3.1.4 Selekcja typu onlooker (wagi rankingowe)

W fazie onlookerów wybór modyfikowanego źródła odbywa się na podstawie *rangi* rozwiązania w populacji (a nie surowych wartości funkcji celu), co zapobiega dominacji jednego źródła. Procedura

wag rankingowych jest następująca:

1. Posortuj populację malejąco wg krotki porządkowej (patrz Section 3.1).
2. Rozwiązaniu o randze r (od 0) przypisz wagę $w_r = N - r$.
3. Losuj indeks źródła z rozkładem proporcjonalnym do w_r .

Liczba prób onlookerów w iteracji wynosi O (parametr; domyślnie $O = N$).

3.1.5 Faza zwiadowcza (scout)

Jeśli licznik nieudanych prób (trial) dla rozwiązania przekroczy limit L , rozwiązanie jest zastępowane nowym losowym rozwiązaniem dopuszczalnym (Algorithm 2 + Algorithm 3). Po udanym zastąpieniu licznik prób się zeruje. Jeżeli nie udaje się wygenerować nowego rozwiązania w zadanej liczbie prób, licznik również się zeruje, by nie zapętlać próby zastąpienia.

3.1.6 Pseudokod operatora mutacji

Algorytm 4 MUTACJAWAYPOINTOW

Wejście: rozwiązanie s , maxAttempts

Wyjście: rozwiązanie s' (jeśli udało się znaleźć dopuszczalnego sąsiada) lub $\perp$

```

1: dla  $j = 1, \dots, \text{maxAttempts}$  wykonuj
2:    $\mathbf{W}' \leftarrow s.\mathbf{W}$ 
3:    $\rho \leftarrow \text{rand}(0, 1)$ 
4:   jeżeli  $\rho < 0,30$  wtedy ▷ Dodaj
5:     dobierz kandydata  $a$  ze zbioru top-20 wg  $(v_a + 1)/(\tau_{t_a} + 1)$ 
6:     jeżeli liczność typu  $t_a$  po dodaniu  $\leq M_{t_a}$  wtedy
7:       wstaw  $a$  w losowe miejsce  $\mathbf{W}'$ 
8:     w przeciwnym razie
9:       continue
10:    koniec jeżeli
11:  w przeciwnym razie jeżeli  $\rho < 0,55$  wtedy ▷ Usuń
12:     $R \leftarrow$  pozycje, których usunięcie nie złamie  $m_t$ 
13:    jeżeli  $R = \emptyset$  wtedy continue
14:    koniec jeżeli
15:    usuń element o losowej pozycji z  $R$ 
16:  w przeciwnym razie jeżeli  $\rho < 0,75$  wtedy ▷ Zamień
17:    jeżeli  $|\mathbf{W}'| < 2$  wtedy continue
18:    koniec jeżeli
19:    wylosuj  $i, k$ ; zamień  $\mathbf{W}'[i] \leftrightarrow \mathbf{W}'[k]$ 
20:  w przeciwnym razie jeżeli  $\rho < 0,95$  wtedy ▷ 2-opt
21:    jeżeli  $|\mathbf{W}'| < 4$  wtedy continue
22:    koniec jeżeli
23:    wylosuj  $i \leq k$ ; odwróć  $\mathbf{W}'[i..k]$ 
24:  w przeciwnym razie ▷ Przesuń
25:    jeżeli  $|\mathbf{W}'| < 2$  wtedy continue
26:    koniec jeżeli
27:    wylosuj  $i, k$ ; przenieś  $\mathbf{W}'[i]$  na pozycję  $k$ 
28:  koniec jeżeli
29:   $P \leftarrow \text{BUDUJSCIEZKEZWAYPOINTOW}(\mathbf{W}')$ 
30:  jeżeli  $P \neq \perp$  i ocena  $P$  daje rozwiązanie dopuszczalne wtedy
31:    zwróć rozwiązanie zbudowane z  $\mathbf{W}'$  i  $P$ 
32:  koniec jeżeli
33: koniec dla
34: zwróć  $\perp$ 

```

3.1.7 Funkcja oceny ścieżki

 Funkcja `evaluate_path` (plik `initial_population.py`) liczy:

- `movement_cost` — $\sum_l w_{p_l} \cdot \delta_l$,
- `attraction_cost` — suma k_i po unikalnych odwiedzonych atrakcjach,
- `total_time` — `movement_cost` + $\sum \tau_{t_i}$,
- `total_value` — $\sum v_i$ po unikalnych odwiedzonych atrakcjach,
- `type_counts` — wektor $[\sum_{i:t_i=t} \mathbb{1}_{a_i \in \mathcal{V}(P)}]_{t=0, \dots, T-1}$,
- flagi: `budget_ok` ($\leq B$), `time_ok` ($\leq T_{\max}$), `type_ok` ($m_t \leq \cdot \leq M_t$) i `feasible` (koniunkcja).

Spójność z modelem matematycznym jest zapewniona: zwracana krotka cech odpowiada wprost ograniczeniom (6)–(7).

3.2 Podejścia pomocnicze

W kodzie znajdują się także uproszczone metody pomocnicze, używane głównie do testowania interfejsu wizualizacji w aplikacji Streamlit.

3.2.1 Random Walk

Algorytm losowy spaceruje od s po sąsiedztwie 8-kierunkowym, dopóki nie zostanie wykorzystany budżet kosztu lub nie zostanie osiągnięty punkt e . Nie pilnuje ograniczeń typów, nie unika powtarzania komórek/krawędzi i służy wyłącznie jako baseline. Implementacja: `src/path_solver.py`, funkcja `solve_random_walk`.

3.2.2 Greedy attractions

W każdym kroku wybierana jest najbliższa (w metryce Chebysheva) jeszcze nieodwiedzona atrakcja i algorytm wykonuje kroki w jej kierunku, dopóki budżet pozwala. Implementacja: `src/path_solver.py`, funkcja `solve_greedy_attractions`. Także baseline.

3.3 Inne pomysły rozważane w fazie projektowej

W trakcie projektu rozważano kilka alternatywnych dróg, które nie weszły do finalnej implementacji:

- **Algorytm genetyczny (GA)** — z populacją chromosomów jako permutacji wybranych atrakcji; krzyżowania typu PMX/OX, mutacje swap/insert/2-opt. Architektura aplikacji (`app.py`) zakłada możliwość podłączenia GA przez stabilny interfejs `solve(map, params) → path`, więc rozszerzenie jest naturalne.
- **Programowanie liniowe całkowitoliczbowe (ILP)** — model (1)–(11) można sprowadzić do mieszanej formulacji ILP z przepływami w grafie siatki; rozważano użycie solvera (np. CBC) do małych instancji jako wartość referencyjna. Decyzja: pominięto ze względu na koszt implementacji i ograniczoną skalowalność.
- **Symulowane wyżarzanie (SA)** — jako pojedyncze-rozwiązaniowa alternatywa, działająca na tej samej reprezentacji waypointów i tym samym zbiorze operatorów. Pomysł porzucono, ponieważ ABC dawało już zadowalające wyniki, a równoległa populacja jest wartością samą w sobie (np. do dalszego ranking-u/eksploracji).
- **Ant Colony Optimization (ACO)** — pomysł na klasycznej stronie konstrukcji ścieżki, jednak nieoczywista adaptacja do twardych ograniczeń typów; pominięto.

W niniejszej dokumentacji szczegółowo opisany jest tylko algorytm ABC, gdyż tylko on doczekał się pełnej implementacji.

4 Aplikacja — dokumentacja użytkownika

Aplikacja jest zestawem narzędzi w języku Python (3.10+), uruchamianych z wiersza poleceń, oraz pomocniczej aplikacji webowej zbudowanej w bibliotece Streamlit. Dwie ścieżki pracy uzupełniają się:

- **Tryb CLI** — generacja mapy, generacja populacji metodą ABC, walidacja i wizualizacja.
- **Tryb web (Streamlit)** — interaktywny edytor/generator mapy, podgląd, prostsze algorytmy (greedy/random) i moduł porównawczy wyników.

4.1 Wymagania i instalacja

Wymagania: Python ≥ 3.10 , biblioteki: `numpy`, `matplotlib`, `pandas`, `streamlit`. Pełna lista znajduje się w pliku `requirements.txt`:

```
1 streamlit>=1.35.0
2 pandas>=2.0.0
3 matplotlib>=3.8
4 numpy>=1.26
```

Listing 1: Zawartość pliku `requirements.txt`

Instalacja w środowisku wirtualnym:

```
1 python -m venv .venv
2 # Windows:
3 .venv\Scripts\activate
4 # Linux/Mac:
5 source .venv/bin/activate
6
7 pip install -r requirements.txt
```

4.2 Charakterystyka danych

4.2.1 Format mapy — CLI (`map.json`)

Mapa generowana skryptem `map_generator.py` zapisywana jest do pliku JSON. Schemat:

```
1 {
2   "weight": [[int, ...], ...], // siatka H x W wag (int >= 1)
3   "time": int, // budżet czasu T_max
4   "budget": int, // budżet pieniężny B
5   "attractions": [
6     [row, col, value, cost, type], // atrakcja: 5 intów
7     ...
8   ],
9   "attraction_types": [
10    [time, min, max], // typ: (czas zwiedzania, min, max)
11    ...
12  ]
13 }
```

Listing 2: Schemat `map.json`

Uwaga. Atrakcje są listami pięciu intów; `type` to indeks (0-based) typu w tablicy `attraction_types`.

4.2.2 Format mapy — aplikacja web (`scenarios/*.json`)

Aplikacja Streamlit posługuje się rozszerzonym formatem (m.in. z nazwą, nazwami typów, polami startu/końca jako słowniki):

```
1 {
2   "name": "Small 8x8 - 4 atrakcje",
3   "width": 8, "height": 8,
4   "weights": [[3,2,4,...], ...],
5   "attractions": [
6     {"id": "a1", "x": 1, "y": 5, "value": 5, "type": "zabytek"},
7     ...
8   ],
9   "attraction_types": ["zabytek", "przyroda"],
10  "start": {"x": 0, "y": 0},
11  "end": {"x": 7, "y": 7},
12  "budget": 30,
```

```

13  "seed": 7
14  }

```

Listing 3: Schemat scenarios/*.json (wycinek z small.json)

4.2.3 Format populacji (initial_population.json)

```

1  [
2  {
3      "id": 0,
4      "path": [[r, c], [r, c], ...],
5      "visited_attractions": [[r, c], ...],
6      "used_attractions": [[r, c], ...],
7      "movement_cost": float,
8      "attraction_cost": int,
9      "total_time": float,
10     "total_value": int,
11     "type_counts": [int, ...],
12     "budget_ok": bool, "time_ok": bool, "type_ok": bool,
13     "feasible": bool
14 },
15 ...
16 ]

```

4.3 Uruchamianie — tryb CLI

4.3.1 Krok 1. Wygenerowanie mapy

Skrypt `map_generator.py` po uruchomieniu wprost generuje mapę 30×30 z trzema typami atrakcji i zapisuje wynik do `map.json`:

```
1 python map_generator.py
```

Domyślne wartości (do edycji w bloku `if __name__ == "__main__":`):

- rozmiar: 30×30 ,
- zakres wag pól: $[7, 12]$,
- budżet czasowy: 800, budżet pieniężny: 300,
- trzy typy atrakcji z różnymi parametrami (τ, m, M) i zakresami wartości oraz kosztów.

4.3.2 Krok 2. Generowanie populacji metodą ABC

```

1 python initial_population.py \
2     --population-size 50 \
3     --iters 200 \
4     --limit 60 \
5     --out initial_population.json

```

Parametry skryptu:

- map** ścieżka do pliku mapy JSON (domyślnie `map.json`).
- out** plik wyjściowy populacji (domyślnie `initial_population.json`).
- population-size** rozmiar populacji N (domyślnie 50).
- iters** liczba iteracji ABC (domyślnie 200).

--limit limit stagnacji L uruchamiający fazę zwiadowczą (domyślnie 60).
--onlookers liczba prób onlookerów na iterację (domyślnie = N).
--seed ziarno generatora liczb losowych (opcjonalne).

Wyjście: plik JSON z populacją oraz wypisane na stdout statystyki populacji (Listing 4) i „najlepsza trasa”.

```

1 === STATYSTYKI POPULACJI POEZATKOWEJ ===
2 Liczba tras: 50
3 Spelnia wszystko: 50/50
4 Spelnia budzet: 50/50
5 Spelnia limit czasu: 50/50
6 Spelnia ograniczenia typow: 50/50
7
8 Sredni calkowity czas: ...
9 Sredni koszt ruchu: ...
10 Sredni uzyty budzet: ...
11 Sredni a liczba atrakcji: ...
12 Srednia wartosc atrakcji: ...
13 Srednia dlugosc sciezki (liczba pol): ...
14
15 Srednia liczba atrakcji kazdego typu:
16   Typ 0: ... (min=..., max=..., time=...)
17   ...

```

Listing 4: Przykład wyjścia statystyk populacji (`print_population_stats`).

4.3.3 Krok 3. Walidacja wygenerowanej populacji

Skrypt `examples/validate_population.py` sprawdza, czy:

- każda trasa ma flagę `feasible=true`,
- żadna krawędź (nieskierowana) nie występuje w trasie dwa razy.

```
1 python examples/validate_population.py initial_population.json
```

Zwraca kod wyjścia 0 w razie pomyślnej walidacji.

4.3.4 Krok 4. Wizualizacja tras

```
1 python visualize_paths.py
```

Domyślnie funkcja `draw_paths` czyta `map.json` i `initial_population.json`. Można wybrać podzbiór tras po id:

```

1 from visualize_paths import draw_paths
2 draw_paths(selected_ids=[0, 1, 5, 10])

```

Lub wskazać inny plik populacji:

```

1 python -c "from visualize_paths import draw_paths;
   draw_paths(population_file='initial_population_abc.json')"

```

Wynik: okno Matplotlib z wizualizacją (atrakcje jako czarne punkty, ścieżki w różnych kolorach, start jako zielony kwadrat, koniec jako czerwony X).

4.4 Uruchamianie — aplikacja web (Streamlit)

```
1 streamlit run app.py
```

Aplikacja udostępnia dwa tryby (wybierane w bocznym pasku):

4.4.1 Tryb „Edytor / Generator”

W lewej kolumnie znajduje się formularz generatora mapy oraz edytor parametrów (rozmiar, liczba atrakcji, ziarno, zakresy wartości i wag, typy atrakcji, rozkład wag — `uniform/clusters`, budżet, punkty startu i końca). Po wygenerowaniu można dalej ręcznie edytować mapę (nazwę, budżet, listę atrakcji w tabeli) oraz wczytywać i zapisywać scenariusze (`scenarios/*.json`).

W prawej kolumnie widoczna jest mapa z wagami w kolorach. Można uruchomić jeden z dwóch baselineów:

- **Greedy (zachłanny)** — `solve_greedy_attractions`,
- **Random walk** — `solve_random_walk`.

Pod wynikiem prezentowane są metryki: koszt, budżet, wartość, liczba odwiedzonych atrakcji oraz informacja, czy ścieżka mieści się w budżecie.

4.4.2 Tryb „Wyniki”

Pozwala wczytać plik wyników o strukturze:

```

1 {
2   "map_file": "scenarios/small.json",
3   "runs": [
4     {
5       "algo": "GA",
6       "path": [[x, y], ...],
7       "history": [fitness_0, fitness_1, ...],
8       "fitness": liczba
9     },
10    ...
11  ]
12 }
```

W zakładkach prezentowane są: *mapa z nałożonymi ścieżkami*, *wykres konwergencji* (na podstawie `history`) i *tabela zbiorcza* (`algorithm`, `fitness`, `koszt`, `wartość`, `liczba atrakcji`, `długość ścieżki`, flaga *feasible*).

4.5 Ustawianie parametrów instancji

Parametry instancji ustawiane są w zależności od ścieżki użycia:

- **CLI** — bezpośrednio w pliku `map_generator.py` (zmienne `rows`, `cols`, `time_limit`, `budget`, wywołania `add_attraction_type`), a następnie uruchomienie `python map_generator.py`.
- **Web** — formularz w lewej kolumnie trybu „Edytor / Generator” (rozmiar, liczba atrakcji, zakresy wartości i wag, typy, dystrybucja, start/koniec, budżet, `seed`). Można też wczytać gotowy scenariusz z `scenarios/` lub z uploadu pliku.

4.6 Ustawianie parametrów algorytmu (ABC)

Parametry sterujące zachowaniem ABC są dostępne jako argumenty linii poleceń (patrz wcześniejszy podrozdział). W praktyce:

- Zwiększenie `--iters` i `--onlookers` daje lepszą eksploatację, ale wydłuża czas.
- Niższy `--limit` sprzyja eksploracji (częstsze użycie zwiadowców), wyższy — eksploatacji.
- `--seed` zapewnia powtarzalność wyników.

W kodzie funkcja `generate_abc_population` przyjmuje też dodatkowy parametr `deduplicate` (domyślnie `True`), pilnujący, by populacja końcowa nie zawierała duplikatów ścieżek.

4.7 Wyniki i sposób ich interpretacji

Statystyki agregowane. Funkcja `print_population_stats` wypisuje licznosc rozwiązań spełniających poszczególne ograniczenia (budżet, czas, typy, wszystko), średnie wartości metryk i rozkład licznosci typów. Pomaga to ocenić „zdrowie” populacji (np. niski odsetek `feasible` może oznaczać zbyt rygorystyczne ograniczenia instancji).

Najlepsza trasa. Skrypt wyświetla cechy najlepszego rozwiązania wg leksykograficznej krotki porządkowej (Value $\uparrow$, Time $\downarrow$, movement cost $\downarrow$, attraction cost $\downarrow$).

Wizualizacja. W oknie `visualize_paths.py`:

- czarne punkty — atrakcje (każda atrakcja niezależnie od typu),
- linie kolorowe — poszczególne trasy z populacji,
- zielony kwadrat — start,
- czerwony X — koniec.

Im więcej linii „konverguje” do podobnego kształtu, tym silniejsza była selekcja w ABC (mniejsza różnorodność końcowa). Jeśli linie są mocno różne i rozproszone, populacja jest zróżnicowana — co bywa pożądane jako materiał wejściowy dla dalszej optymalizacji.

Walidacja. Skrypt `examples/validate_population.py` sygnalizuje:

- `bad_no_repeated_edges` — liczba tras łamiących zakaz powtarzania krawędzi (oczekiwane: 0),
- `infeasible` — liczba rozwiązań niedopuszczalnych (oczekiwane: 0).

Sumarycznie wypisuje status OK lub NOT OK.

5 Eksperymenty

W bieżącej wersji projektu wdrożony jest mechanizm zbierania metryk populacji (`print_population_stats` w `initial_population.py`) oraz walidator `examples/validate_population.py`, co stanowi podstawę pod planowane eksperymenty. Pełna seria pomiarów porównawczych nie została jeszcze przeprowadzona — w tym rozdziale prezentujemy jedynie zarys planu eksperymentalnego oraz dane, jakie są możliwe do uzyskania z aktualnego kodu.

5.1 Plan eksperymentalny

Zmienne niezależne.

- Rozmiar mapy: $H \times W \in \{10 \times 10, 20 \times 20, 30 \times 30\}$.
- Gęstość atrakcji: liczba atrakcji $\in \{4, 15, 30\}$.
- Rozkład wag: `uniform` vs. `clusters`.
- Parametry ABC: $N \in \{20, 50, 100\}$, $I \in \{50, 200, 500\}$, $L \in \{30, 60, 100\}$, $O \in \{N/2, N, 2N\}$.
- Ziarno losowości `seed` — minimum 5 powtórzeń dla każdej kombinacji.

Zmienne zależne (mierzone).

- Frakcja rozwiązań dopuszczalnych w populacji końcowej.
- Średnia, mediana i maksimum `total_value` w populacji.
- Średni `movement_cost`, `total_time`, `attraction_cost`.

- Średnia długość ścieżki (liczba pól).
- Liczba unikalnych ścieżek (różnorodność populacji).
- Czas wykonania (sekundy) jako funkcja N , I , O .

Hipotezy do weryfikacji.

1. Zwiększenie I (liczby iteracji) podnosi medianę `total_value`, ale z malejącym przyrostem (krzywa nasycenia).
2. Faza onlookerów wzmacnia eksploatację — wyższe O przy stałym I poprawia średnią wartość celu, ale zmniejsza różnorodność populacji.
3. Wyższy limit stagnacji L zwiększa eksploatację kosztem eksploracji; bardzo niskie L powoduje nadmierne „resetowanie” przez zwiadowców i uniemożliwia dopracowanie rozwiązań.
4. Rozkład wag `clusters` (klastry niskokosztowe) daje wyższe wartości `total_value` w tym samym budżecie czasu niż `uniform`.

5.2 Porównanie z baselineami

W kodzie dostępne są dwa proste algorytmy referencyjne: `solve_random_walk` i `solve_greedy_attractions` (`src/path_solver.py`). Porównawcza tabela ma postać:

Tabela 2: Szablon zestawienia wyników (do uzupełnienia).

Algorytm	value	move_cost	total_time	feasible
Random Walk	—	—	—	—
Greedy attractions	—	—	—	—
ABC (best in pop)	—	—	—	—
ABC (mean in pop)	—	—	—	—

5.3 Zbieranie wyników

Aktualny stan kodu pozwala uzyskać następujące dane bez modyfikacji implementacji:

- Surowe statystyki (Listing 4 z Section 4) — wypisywane na stdout po uruchomieniu `initial_population.py`
- Plik `initial_population.json` z wszystkimi rozwiązaniami — pozwala policzyć dowolne dodatkowe statystyki ex post (np. w notatniku).
- Wizualizację (`visualize_paths.py`) dla wybranych id.

5.4 Wyniki wstępne

Pełne wyniki eksperymentalne zostaną dołączone w kolejnej wersji dokumentacji. Bieżąca implementacja przeszła testy poprawności:

- Wszystkie wygenerowane rozwiązania w populacji końcowej spełniają wszystkie ograniczenia (`feasible=true`) — co jest gwarantowane konstrukcyjnie przez algorytm.
- Walidator `validate_population.py` potwierdza brak powtórzeń krawędzi w każdej trasie.
- Wizualizacja prezentuje sensowne kształty tras (krzywe biegnące przez atrakcje, omijające zablokowane atrakcje niezaplanowane).

6 Podsumowanie

6.1 Wnioski

1. Postawiony problem łączy aspekty kombinatoryczne (wybór podzbioru atrakcji z ograniczeniami typów i budżetów) z grafowymi (istnienie i koszt fizycznej ścieżki na siatce 8-kierunkowej z zakazem powtórzeń krawędzi). Tradycyjne sformułowania *Orienteering Problem* stanowią dobrą inspirację, ale twarde wymaganie braku powtórzeń krawędzi czyni nasz model wyraźnie trudniejszym.
2. Dwupoziomowa reprezentacja rozwiązania (waypointy + materializacja A^*) okazała się praktyczna — pozwala rozdzielić problem „co odwiedzić” od „jak dojść” i drastycznie zawężyła przestrzeń sąsiedztwa.
3. Algorytm ABC daje populację rozwiązań w pełni dopuszczalnych, co eliminuje konieczność funkcji kary i upraszcza analizę.
4. Leksykograficzna krotka porządkowa (Value $\uparrow$, Time $\downarrow$, Cost_{ruch} $\downarrow$, Cost_{atrakcji} $\downarrow$) jest tanim, ale skutecznym kryterium różnicowania rozwiązań i kierowania eksploatacją.

6.2 Napotkane problemy

- **Trudność generacji rozwiązań dopuszczalnych.** W instancjach o ciasnym budżecie czasowym lub bardzo dużej liczbie atrakcji częstość uzyskiwania ścieżek dopuszczalnych przy losowych waypointach jest mała — konieczne jest podbicie liczby prób (`max_tries`) w `_random_feasible_solution`.
- **Twardy zakaz powtórzeń krawędzi.** Wprowadzenie tego ograniczenia powoduje, że pewne kombinacje waypointów po prostu nie da się fizycznie zrealizować (segment domyka się dopiero po długim „objeździe”); musi to być uwzględnione w mutacjach jako naturalny kanał odrzuceń.
- **Koszt A^* przy każdej mutacji.** Każda zaakceptowana mutacja oznacza materializację co najmniej jednego segmentu ścieżki; przy populacji 50 i 200 iteracjach koszt ten kumuluje się szybko — optymalizacja A^* (np. z cachowaniem segmentów lub heurystyką niższej jakości, ale szybszą) jest naturalnym kierunkiem przyspieszania.

6.3 Kierunki rozwoju

1. **Pełna integracja ABC z aplikacją Streamlit.** Architektura `app.py` przewiduje stabilny interfejs `solve(map, params) → path`; dorzucenie wrappera nad `generate_abc_population` pozwoli porównywać ABC z baselineami w UI.
2. **Drugi algorytm meta-heurystyczny.** GA / SA / Tabu Search na tej samej reprezentacji waypointów (operatory już mamy) dałyby porównanie wewnątrz tej samej klasy metod.
3. **Solver ILP referencyjny.** Dla małych instancji (np. 10×10 , 6–8 atrakcji) sformułowanie z rozdziału 2.2 można zlecić solverowi — daje „prawdziwie optymalną” wartość celu i pozwala obiektywnie ocenić jakość ABC.
4. **Bardziej wyrafinowana funkcja oceny.** Wprowadzenie metryki Pareto-typu (niezdominowana wartość vs. czas) zamiast leksykografii, by jawnie zwracać front Pareto.
5. **Cache segmentów A^* .** Ponieważ ABC modyfikuje pojedyncze pozycje waypointów, większość segmentów ścieżki pozostaje niezmienna — ich przeliczanie można zaoszczędzić.
6. **Większa baza scenariuszy testowych.** Aktualnie dostępne są dwa scenariusze webowe (`small.json`, `medium.json`); benchmark wymaga większej i bardziej zróżnicowanej kolekcji.

7 Literatura

Literatura

- [1] D. Karaboga, *An Idea Based on Honey Bee Swarm for Numerical Optimization*, Technical Report TR06, Erciyes University, 2005.
- [2] D. Karaboga, B. Basturk, *A powerful and efficient algorithm for numerical function optimization: Artificial Bee Colony (ABC) algorithm*, Journal of Global Optimization, 39(3), 2007.
- [3] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden, *The Orienteering Problem: A survey*, European Journal of Operational Research, 209(1), 2011.
- [4] P. E. Hart, N. J. Nilsson, B. Raphael, *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*, IEEE Transactions on Systems Science and Cybernetics, 4(2), 1968.
- [5] Dokumentacja Pythona 3, <https://docs.python.org/3/>.
- [6] Streamlit Documentation, <https://docs.streamlit.io/>.
- [7] J. D. Hunter, *Matplotlib: A 2D Graphics Environment*, Computing in Science & Engineering, 9(3), 2007; <https://matplotlib.org/>.
- [8] C. R. Harris et al., *Array programming with NumPy*, Nature, 585, 2020; <https://numpy.org/>.

8 Dodatek — podział pracy

Poniższa tabela prezentuje szczegółowy podział zadań w projekcie wraz z procentowym udziałem członków zespołu. Ze względu na charakter przedmiotu (Badania Operacyjne) podział został tak skonstruowany, aby każdy z członków zespołu uczestniczył w realnej pracy algorytmicznej w porównywalnym wymiarze, przy czym **rdzeń metaheurystyki ABC** pozostaje głównym obszarem odpowiedzialności jednego autora.

Etap realizacji	Zakres pracy	Osoba / % udział
Model matematyczny problemu	Formalizacja jako ILP: zmienne x_{ij}, y_i , funkcja celu, ograniczenia (2)–(11), ograniczenia typowe (7)	Adrian 50%, Mikołaj 30%, Bartosz 20%
Algorytm ABC — rdzeń metaheurystyki	<code>abc_algorithm.py</code> : faza employed / onlooker / scout, ruletka selekcji proporcjonalnej, licznik prób, reset rozwiązań, pętla główna i kryterium stopu	Bartosz 70% , Adrian 15%, Mikołaj 15%
Operatory zaburzeń	Operatory mutacji rozwiązania (add / remove / swap / 2-opt / shift) wraz z rozkładem prawdopodobieństw oraz mechanizm utrzymywania dopuszczalności po mutacji	Bartosz 65% , Mikołaj 20%, Adrian 15%

Etap realizacji	Zakres pracy	Osoba / % udział
A* i konstrukcja ścieżek	Implementacja A* na siatce 8-kierunkowej, twardy zakaz powtarzania krawędzi, dynamiczne blokowanie atrakcji spoza listy waypointów, sklejanie segmentów trasy	Adrian 55%, Bartosz 30%, Mikołaj 15%
Funkcja ewaluacji	<code>evaluate_path</code> : wartość trasy, koszt ruchu, koszt atrakcji, czas; leksykograficzne kryterium porównawcze rozwiązań	Adrian 60%, Bartosz 25%, Mikołaj 15%
Generator mapy	<code>map_generator.py</code> , <code>src/map_generator.py</code> : proceduralne generowanie wag terenu, rozmieszczanie atrakcji, przypisanie typów, parametry rozkładu	Mikołaj 65% , Adrian 20%, Bartosz 15%
Populacja początkowa	<code>initial_population.py</code> : heurystyczna konstrukcja rozwiązań dopuszczalnych, dobór punktów pośrednich z uwzględnieniem ograniczeń typowych, kontrola czasu/budżetu	Mikołaj 45%, Bartosz 40%, Adrian 15%
Walidator ograniczeń	<code>examples/validate_population.py</code> : sprawdzanie spełnienia (2)–(11), weryfikacja braku powtórzeń krawędzi i ograniczeń typowych	Adrian 65%, Mikołaj 25%, Bartosz 10%
Parser i I/O	<code>parser.py</code> , <code>load_json.py</code> : konwersja JSON ↔ struktury wewnętrzne, walidacja schematu wejścia	Mikołaj 55%, Adrian 35%, Bartosz 10%
Wizualizacja tras	<code>visualize_paths.py</code> : rysowanie tras, atrakcji, punktów start/end, kolorowanie typów, wykresy diagnostyczne	Mikołaj 70%, Adrian 20%, Bartosz 10%
Aplikacja web (Streamlit)	<code>app.py</code> , <code>src/path_solver.py</code> , <code>src/ui_io.py</code> , <code>src/ui_viz.py</code> : edytor mapy, generator, walidacja, tryb wyników	Mikołaj 50%, Adrian 35%, Bartosz 15%
Scenariusze testowe	<code>scenarios/small.json</code> , <code>scenarios/medium.json</code> : parametry, dobór trudności, dane wejściowe do eksperymentów	Adrian 45%, Mikołaj 35%, Bartosz 20%

Etap realizacji	Zakres pracy	Osoba / % udział
Dokumentacja LaTeX	Niniejszy dokument: opis problemu, model matematyczny, opis algorytmu, dokumentacja użytkownika, literatura	Bartosz 20%
		Adrian 40%, Mikołaj 35%,
		Bartosz 25%

Sumaryczny rozkład wkładu

Po zsumowaniu wkładów z poszczególnych etapów (z wagami proporcjonalnymi do nakładu pracy w danym module) orientacyjny rozkład wkładu autorów w całość projektu przedstawia się następująco:

Autor	Łączny wkład
Bartosz Tokarz	$\approx 34\%$
Adrian Michalski	$\approx 34\%$
Mikołaj Kaleta	$\approx 32\%$

Komentarz do podziału. Bartosz Tokarz odpowiada za rdzeń metaheurystyki ABC — najbardziej wymagający koncepcyjnie i obliczeniowo komponent projektu (główna pętla, operatory zaburzeń, mechanizm scout). Adrian Michalski skupia się na komponentach algorytmicznych powiązanych ze ścieżkami i poprawnością rozwiązań: A^* z ograniczeniami, funkcja ewaluacji, walidator i formalizacja modelu matematycznego. Mikołaj Kaleta odpowiada za algorytmikę po stronie środowiska: proceduralny generator mapy, konstrukcja populacji początkowej oraz warstwa wizualizacji i I/O. Taki podział utrzymuje porównywalny wkład algorytmiczny wszystkich członków zespołu, przy jednoznacznym przypisaniu odpowiedzialności za główny algorytm projektu.